CELESTIAL MECHANICS × AI · 2026

# From the next token to the next theorem

How a model works — and how I use it in practice

Bowen Liu  Draft v0.1  45 min

# What the model sees: ten integers

EN · ONE SENTENCE

Celestial mechanics is the poetry of orbits.

| Cel | estial | _mechanics | _is | _the | _poetry | _of | _or | bits | . |
|---|---|---|---|---|---|---|---|---|---|
| 69126 | 90010 | 52347 | 382 | 290 | 38153 | 328 | 503 | 24043 | 13 |

**46** characters → **10** tokens → 10 integers · _ marks a leading space

CHINESE · FOUNDATIONS OF CELESTIAL MECHANICS

本书系统地介绍了天体力学

| 本 | 书 | 系统 | 地 | 介绍 | 了 | 天 | 体 | 力 | 学 |
|---|---|---|---|---|---|---|---|---|---|
| 4087 | 20121 | 31936 | 5557 | 37887 | 4531 | 867 | 7074 | 11343 | 8590 |

**12** characters → **10** tokens → 10 integers · two-character words often map to one token

“This book systematically introduces celestial mechanics.”

01 Where do these integers come from?

02 How does it predict the next one?

03 How do we put it to work?

# Put the model back in the system




Inner ring: one call · Middle ring: across calls · Outer ring: engineering boundary

| | | |
|---|---|---|
| 01 | **LLM** | A probability generator: tokens in, a distribution over the next token out |
| 02 | **Prompt** | This call's conditions: system / user / few-shot |
| 03 | **Context** | Every token visible to this call; the window is finite |
| 04 | **Memory** | State beyond the window: write / retrieve / compress / forget |
| 05 | **Agent loop** | Observe → act → observe; state lives outside |
| 06 | **Harness** | Tools, sandbox, permissions, verification, logs |

> The model is the CPU; prompt, context, memory, loop and harness are the runtime.

# Acceleration: from basic use to gold medals

2023 · BEFORE

## Basic use

- Explain concepts, search the literature, write code
- Do simple calculations, often wrong
- Complex proofs still need people
- Role: assistant

→ 19 months

2025 · AFTER

## Research level

- IMO 2025 gold level 35/42
- Lean proofs, machine-checkable
- OpenAI 2026: 372 problems, ≈3 h each (report)
- Role: candidate generator

2024.07 **28/42** silver → 2025.07 **35/42** gold level — 12 months, +7 points

> This is a staircase with **three steps**.

# Timeline: three steps

2023–24

## Tools: search + verification

2023.12
**FunSearch**
LLM + evaluator + evolutionary search; a new cap-set construction

2024.01
**AlphaGeometry**
Neuro-symbolic; IMO geometry 25/30

2024.07
**AlphaProof**
Lean + RL; IMO 2024 silver 28/42 (+AG2)

2024–25

## Reasoning: RL + test-time compute

2024.09
**OpenAI o1**
RL teaches thinking; test-time compute

2025.01
**DeepSeek-R1**
RLVR; reasoning behavior emerges; open weights

2025.05
**AlphaEvolve**
Evolutionary coding agent; 4×4 complex matrix product in 48 multiplications

2025–26

## Scale: from benchmarks to open problems

2025.07
**IMO 2025**
Gold level 35/42; natural language, no tools

2025
**Formal gold (claimed)**
Aristotle et al.; Lean-checkable

2026.05–10
**OpenAI**
Unit-distance counterexample; NS / Riemann-related (claimed); 372 problems (report)

> Every leap combines a **generator, a verifier and search**.

# The motor and the assembly line

A story Eric Schmidt tells — and the AI parallel.

When the electric motor was invented, factory owners did the obvious thing: remove the central steam engine, put a motor in its place, and change nothing else. Efficiency rose by about 30%.

Decades later, someone realized the motor could be small. Every machine could have its own. The factory could be designed around the flow of production itself; the power source no longer dictated the layout.

The assembly line was born. Efficiency rose by an order of magnitude.

**30%**

Standalone · Prompting Harder
Resets each session · No compounding

**10×+**

Assembly line · Context Engineering
Persistent assets & rules · Continuous compounding

The gap is **work designed to compound**.

> Models are rented; context and systems are owned. Do not merely prompt in isolation; turn domain rules, tools, and evals into persistent assets.

> The gain is in **layout and compounding**.

# From the chat box to the working directory

Research cannot stay in thin wrappers. Files are interfaces; rules are portable.

L1 · WRAPPER

## Web chat

**Ask, read, manual copy**
No local file access · Ephemeral context

**Zero real tools**
The model suggests; the human executes

L2–3 · COPILOT

## IDE editor

**In-file completions**
Context limited to currently open files

**A faster typewriter**
Same human loop, slightly higher cadence

L4–5 · AGENT LOOP

## Repo & CLI Agent

**Tools & isolated agents**
Symplectic integrators · Series expansion · Reviewers

**Autonomous verification**
Hypothesis → Experiment → Conservation checks → git

- OpenCode
- Claude Code
- Codex CLI
- Cursor
- Markdown SOP Protocol

> Files are interfaces. Build **portable research assets** in local workspaces.

`Swap the foundation model via a single config line; your dynamical check suites remain permanently valid.`

# Two attempts

Same intention. Different governance: from abdication to coaching.

OPENCLAW · EARLY ATTEMPT

### Mistaken for calculator · Hands-off

- Expected the agent to autonomously deliver an end-to-end paper.
- Early model was brittle; hallucinations compounded without evals.
- Complete human abdication, hoping for an unguided miracle.

Lesson: unguided execution collapses deterministically.

CODEX · GPT-5.3–5.5 · ISTART

### Act as Coach · Verification loop

- Postdoc extension; I framed clear goals (WHAT) while AI drafted (HOW).
- Refused to be a line-by-line fixer; coached by flagging flawed steps.
- Caught absurd derivation: $Ax = b,\ b \neq 0 \rightarrow x = 0$ ✕
- After 3 rejections, GPT-6.1 Sol scanned deeper issues for final fixes.

Lesson: verification asymmetry — evaluating is easier than drafting.

> Do not act as a line-by-line fixer; act as a **coach defining standards and counterexamples**.

# Low-hanging fruit & compounding assets

Drafting a paper is only the start; compounding assets are evals and taste.

01 · HUMAN AS COACH

### Frame WHAT & bounds

Define clear objectives; let AI explore the HOW. Point out logical flaws rather than patching code by hand.

02 · SYSTEMATIC EVALS

### Dynamical checks

Models get stuck. Build invariant test suites: Hamiltonian conservation, symplectic symmetry, and residuals.

03 · COMPOUNDING ASSETS

### Taste & eval suites

A single paper depreciates quickly. What compounds is your knowledge base, codebase SOPs, and formal scorers.

OpenAI can brute-force batches of open problems, but old conjectures were written for humans. As marginal output cost hits zero, only **eval systems and contrarian taste** compound.

> The central question: which problems are **worth tackling together with machines**?

# From the next token to the next theorem

`token` → `probability` → `candidate` → `verification` → **`theorem`**

### Probability gives breadth

### Verification gives correctness

### People set the direction

> The next theorem still begins with the **next token**.